Beyond SDN: Empirical Characterization of a Commercial Coding Agent for CLI-Based OSPF Fault Diagnosis

Andrés Duarte Ornelas

October 22, 2026

Abstract

Network fault diagnosis in CLI-managed enterprise environments demands deep protocol expertise that many organizations struggle to maintain, yet no published research has evaluated whether commercial coding agents can perform this work autonomously. This study addresses that gap by evaluating Claude Code's ability to diagnose and resolve OSPF faults through closed-loop SSH-based CLI interaction with no programmatic interfaces available, mirroring the interaction model a human engineer would use. Using a three-router VyOS topology in GNS3 modeled after the CCNA 11.6.3 OSPF troubleshooting lab, the agent was presented with six simultaneously active faults across 50 repeated experimental sessions. The agent achieved an effective fault resolution rate of approximately 97%, a perfect-run rate of approximately 78%, a mean time to resolution of 4 minutes 15 seconds, and a mean per-session cost of $0.40. Notably, no session introduced a regression that left any fault in a worse state than it was found, which is the failure mode of greatest operational consequence in network environments. These results establish a demonstrated capability floor for general-purpose coding agents in traditional CLI-managed networks and provide the first empirical characterization of their failure modes for practitioners assessing deployment risk.

1 Introduction

Network fault diagnosis in enterprise environments has changed little in its fundamental interaction model: an engineer connects to a router or switch via SSH, issues a series of `show` commands, interprets unstructured text output, and applies configuration changes one at a time through a command-line interface. This workflow is the operational reality for the majority of production networks today, regardless of how much the underlying technology has advanced.

The challenge is not the interaction model itself but the expertise it demands. Diagnosing a routing protocol fault requires an engineer who can navigate vendor-specific CLI environments, interpret protocol output that varies by device and configuration state, and systematically isolate root causes across multiple devices simultaneously. For smaller organizations without dedicated network operations staff, this level of specialization is difficult to maintain at the depth that complex, multi-protocol environments require.

Recent developments in large language model agents have introduced a capability that makes them plausible candidates for this kind of work. Tools such as Claude Code can now execute shell commands, read output, and iteratively adapt their approach within a single autonomous session. While these tools are primarily marketed toward software development tasks, the underlying interaction loop they perform, issue a command, interpret the result, and decide on a next action, is structurally identical to the diagnostic workflow a network engineer follows at the CLI.

Existing research on LLM agents in network operations has made meaningful progress, but largely within a constrained scope. Studies evaluating intent-based configuration generation and SDN-oriented agent frameworks have demonstrated that frontier models can translate natural language into structured configurations and interact with programmatic controllers with increasing reliability (Adanza et al., 2025, 2026; Lin et al., 2025; Wang et al., 2024; Wei

et al., 2025). The concentration of research in this area is not surprising: SDN environments expose well-documented REST APIs and Python-based controllers whose structured inputs and outputs closely resemble the software development tasks these agents were built for. The interaction problem, in other words, is one LLMs already know how to solve.

Traditional CLI environments offer no such abstraction. An agent operating in a CLI-managed network must navigate vendor-specific syntax, interpret unstructured text output from `show` commands that varies by device state and configuration, manage privilege levels and configuration modes, and verify that applied changes actually resolved the fault rather than introducing new ones. Unlike SDN environments where the interface is self-describing, CLI-based interaction also requires the agent to be supplied with explicit environmental context, which devices exist, how to reach them, what the topology looks like, before it can do anything at all. Despite traditional CLI environments representing the operational reality for the majority of enterprise networks in production today, no published research has evaluated a commercial, general-purpose coding agent performing this kind of closed-loop interaction against real or emulated network operating systems.

Existing terminal agent benchmarks such as Terminal-Bench evaluate agents on tasks contained within a local environment, compiling code, configuring servers, debugging systems, where the agent operates on files and processes it can directly access (Merrill et al., 2026). Network CLI operations present a structurally different challenge: the agent must reach beyond its local environment via SSH into external devices, authenticate against them, navigate vendor-specific privilege hierarchies, and interpret output it has no filesystem-level access to verify. This distinction helps explain why traditional terminal benchmarks have not addressed network operations, it falls outside the interaction model they were designed to evaluate, and also why the networking literature has not addressed it from the other direction.

The consequences of deploying an inadequately evaluated agent in network operations ex-

tend beyond the failures familiar from software development contexts. When a coding agent introduces a bug into a production application, that application goes down. When a network agent misconfigures a router, everything that depends on that network goes down simultaneously. The failure is not contained to a single service but propagates across every system that relies on the network as its communication backbone.

What makes this particularly consequential is the nature of the failure modes that CLI-based agent interaction specifically introduces. An agent that misparses the output of a `show` command and draws an incorrect diagnostic conclusion, issues commands that partially apply before failing, or fixes one fault while silently introducing a misconfiguration elsewhere does not simply fail to help. It may leave the network in a state that is harder to diagnose than the original fault, because unlike a human engineer who knows exactly what they touched, an agent that introduces a regression and then reports success creates a misleading picture of network state that can extend an outage far beyond its original scope. The risk is not just that the agent underperforms, it is that a poorly characterized agent could actively make things worse in ways that are not immediately visible to the operators who deployed it.

Without empirical characterization of how and where these agents succeed and fail in CLI-managed environments, organizations have no basis for assessing that risk. The question is not whether commercial coding agents will be applied to network operations tasks, but whether the field will have produced the evidence needed to guide those decisions responsibly before deployments occur.

This study addresses that gap directly. Using a three-router GNS3 topology running VyOS, modeled after the CCNA 11.6.3 OSPF troubleshooting lab, this experiment evaluates Claude Code's ability to autonomously diagnose and resolve a set of pre-injected OSPF faults through closed-loop CLI interaction via Netmiko across repeated experimental sessions. The agent is provided with explicit environmental context and interacts with the network devices

exclusively through SSH-based terminal commands, the same interaction model a human engineer would use, with no programmatic interfaces or structured APIs available to it.

The central research question guiding this study is: to what extent can a commercial, general-purpose CLI coding agent autonomously diagnose and resolve multi-fault OSPF scenarios in a traditional, non-SDN network environment, and what patterns of success and failure characterize its behavior across repeated trials? Supporting sub-questions examine correctness outcomes such as fault resolution rate, efficiency outcomes such as token consumption and time to resolution, and behavioral outcomes such as diagnostic strategy, command validity, and regression introduction.

The goal is not to evaluate a purpose-built network AI system but to characterize how a general-purpose commercial tool performs when transferred to a specialized domain it was not explicitly designed for. Both success and failure outcomes carry value: success establishes a demonstrated floor of capability that practitioners can evaluate against their own environments, while failure characterizes the specific risk categories that must be accounted for before any responsible deployment can occur.

2 Methodology

This study employs a quantitative experimental design. The experimental environment consists of three VyOS routers (HQ, Branch1, Branch2) in a triangle topology running inside GNS3, modeled after the CCNA 11.6.3 OSPF troubleshooting lab. Figure 1 shows this arrangement, with each router connected to the other two via dedicated WAN-side interfaces, each serving a distinct LAN segment, and HQ carrying a loopback interface that simulates the upstream internet connection used for default route advertisement.

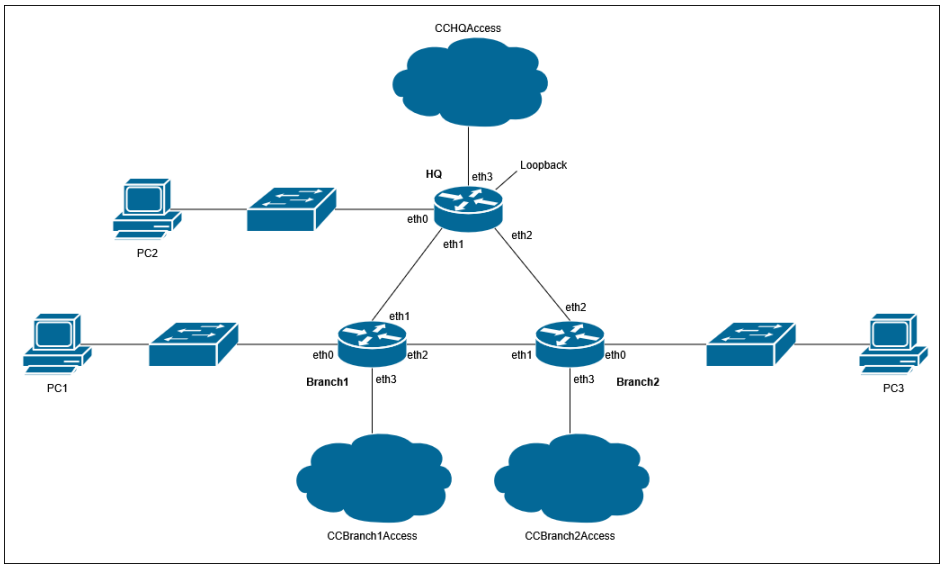


Figure 1: Topology of GNS3 Environment

Each session presents Claude Code with six simultaneously active OSPF faults: an administratively disabled WAN interface, a passive-interface assigned to the wrong interface, an incorrect CIDR prefix length in an OSPF network statement present on two routers, an incorrect IP address on a LAN interface, and a missing `default-information originate` command. Inside WSL, Claude Code (using the Opus 4.6 model with medium effort) interacts with the topology, which is operating in a GNS3 server via Netmiko, issuing SSH-based CLI commands to each router's assigned address within a single agentic session. SSH reachability from the WSL host to each router's management address was provided through Tailscale. The agent was given only the router IP addresses and credentials; no information about the underlying connectivity method was included in its context. This arrangement kept the transport mechanism transparent to the agent, ensuring it could not influence diagnostic reasoning. The specific connectivity method is not a controlled variable in this study: what matters is that uninterrupted SSH access was available for the duration of each session. A standardized prompt initiates each session. Data is collected through a two-phase Netmiko snapshot procedure capturing post-injection fault confirmation, and post-agent-session resolution state, alongside the JSONL session transcript generated by Claude Code under

~/.claude/projects/. Before each session, a reset script restores all routers to their known-broken baseline configuration and confirms all faults are in place. The experiment targets a minimum of fifty complete sessions. Figure 2 illustrates this layered access model, with Claude Code running inside WSL on the Windows host and reaching the GNS3-managed routers via Netmiko over their 192.168.122.x management addresses.

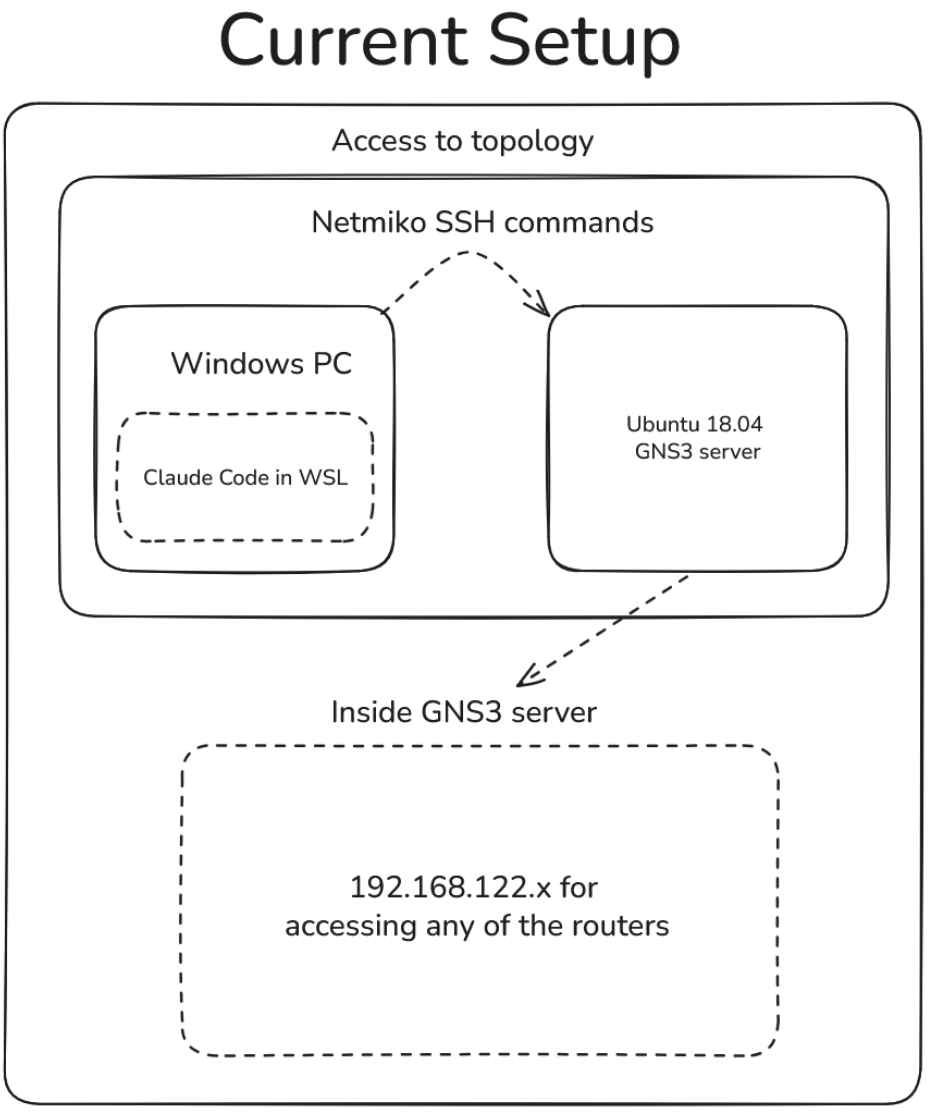


Figure 2: Current Setup Diagram

The independent variable is the initial network state presented to the agent, specifically the fixed set of six simultaneously active OSPF faults. Control variables include the GNS3

topology, VyOS router baseline configurations, the specific fault set, the Claude Code model version (Opus 4.6 with medium effort), and the standardized initialization prompt. Dependent variables are the measured outcomes across three categories: correctness, efficiency, and behavioral.

Each session was initiated using a standardized two-component prompt structure. A `CLAUDE.md` project file provided the agent with environmental context including the network topology, intended addressing table, management access credentials, and diagnostic approach guidance. The session was then initiated with the instruction 'Fix the network.' This structure ensured that all variation in agent behavior across sessions was attributable to the agent's reasoning rather than differences in task framing or available context.

Analysis spans three categories. Correctness outcomes are scored using a rubric derived from the CCNA 11.6.3 answer key, covering binary task completion and partial credit for each discrete resolution step per fault. Efficiency outcomes are summarized using descriptive statistics across sessions for token consumption, CLI command count, compaction events, and elapsed time. Behavioral outcomes are coded from session transcripts, capturing diagnostic strategy, device access sequence, command validity, unnecessary configuration changes, and regression introduction.

Internal validity is maintained through session isolation: each session begins from a confirmed baseline state with the complete fault set freshly applied. Reliability is addressed through fifty repeated trials of the same scenario. Limitations affecting external validity include the three-router VyOS topology, which does not reflect the scale or vendor diversity of production environments; the single-agent scope, which precludes generalization to other tools; and the CCNA pedagogical origin of the fault set, which may be well-represented in the agent's training data and could inflate apparent performance relative to novel, production-realistic fault patterns.

3 Results

Across 50 complete experimental sessions, Claude Code demonstrated strong overall performance on all three outcome categories, with one dominant failure mode that is better characterized as a grading artifact than an agent deficiency. The following subsections report correctness, efficiency, and behavioral findings in turn.

3.1 Correctness Outcomes

The automated scoring rubric evaluated 300 discrete fault resolution instances across the 50 sessions, 6 faults per run, and recorded 255 as correctly resolved for an overall fix rate of 85.0%. Eleven of 50 runs achieved a perfect 6/6 result, yielding a perfect run rate of 22.0%. Thirty-seven runs scored 5/6, and two runs scored 2/6 due to execution-level failures discussed at the end of this section.

Per-fault fix rates reveal a sharply bimodal pattern. Errors #1 and #2 (an incorrect IP address on HQ eth0 and an incorrect OSPF network mask on HQ) were resolved in all 50 runs without exception. Errors #4, #5, and #6 (an administratively disabled WAN interface, a passive-interface misconfiguration, and an incorrect OSPF LAN network statement on Branch2) each achieved a 96% fix rate (48/50). Error #3, the missing `default-information originate` command on HQ, was resolved as scored in only 22% of runs (11/50) and is the dominant failure mode. Figure 3 makes this asymmetry immediately visible: all other errors cluster between 96% and 100% on both rubrics, while Error #3's original bar is the sole outlier, resolving to full parity only after the rubric adjustment.

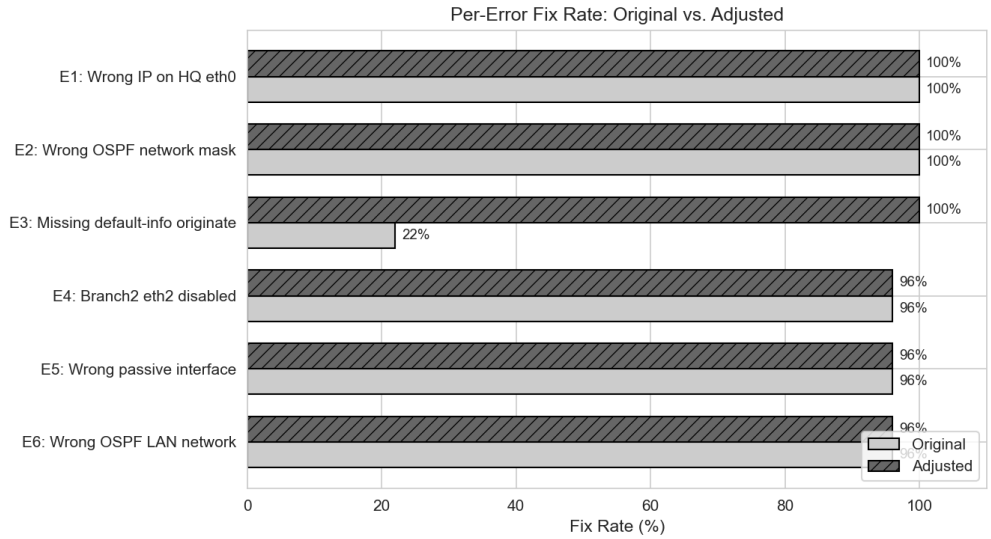


Figure 3: Per Error Fix Rate: Original vs. Adjusted

Figure 4 shows the effect of this adjustment at the run level: the dominant 5/6 cluster of 37 runs under the original rubric collapses almost entirely into the 6/6 bar under the adjusted rubric, with only the two genuine execution failures remaining below a perfect score.

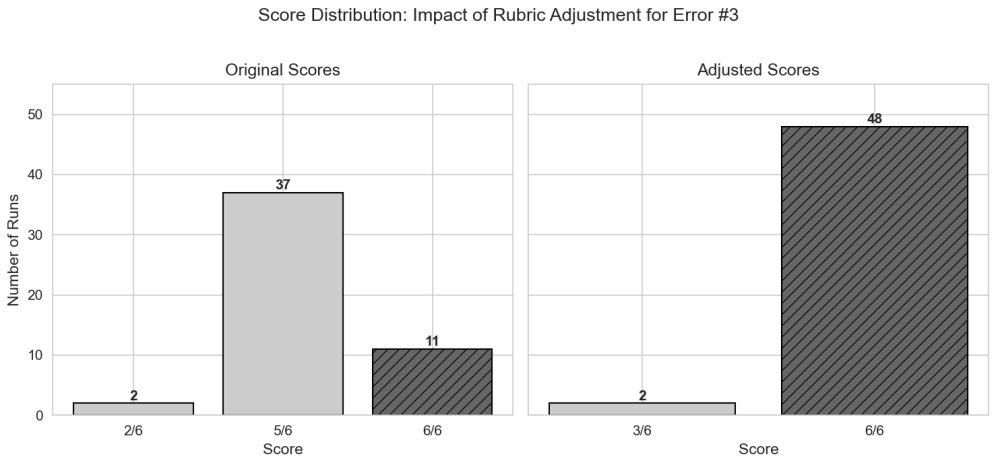


Figure 4: Score Distribution: Impact of Rubric Adjustment for Error #3

This figure, however, reflects a limitation of the scoring rubric rather than a failure of the agent. In 39 of 50 runs, the agent applied the command `set protocols ospf default-information originate always`, a functionally valid variant that causes HQ to advertise a default route

unconditionally rather than conditionally. The rubric, derived from the CCNA 11.6.3 answer key, accepted only the base form of the command and did not account for this variant. Because the applied command resolves the fault and represents a legitimate and arguably more resilient configuration choice, these 39 instances are better characterized as rubric mismatches than agent errors. Accounting for both accepted command forms, the effective fault resolution rate rises to approximately 97%, with a corresponding perfect run rate of approximately 78%.

Branch1 achieved a 100% configuration match rate across all sessions, which is expected given that no faults were injected on that router. HQ achieved a 22% match rate under strict scoring, attributable almost entirely to the `always` variant discussed above, rising to approximately 100% when that variant is accepted. Branch2 registered a 4% match rate against the answer key, reflecting a systematic configuration divergence discussed in the behavioral outcomes section rather than unresolved faults.

The two runs scoring 2/6 (r018 and r023) represent genuine execution failures rather than grading artifacts. In both cases, transcript analysis confirms that the agent correctly identified all six faults, applied correct fix commands, and verified via `show` commands and ping tests that the network was operating correctly before session end. The Branch2 fixes did not persist to the post-session verification snapshot. In r023, a redundant `configure` token prepended to the Netmiko command list produced an `Invalid command: [configure]` error, but this was cosmetically harmless; the remaining commands executed normally and HQ fixes committed correctly. In both runs, HQ fixes survived into verification while Branch2 reverted to its original broken config within approximately two seconds of session end, despite the data plane showing correct behavior at that time. The root cause cannot be confirmed post-hoc, but the most plausible explanation is a Netmiko/configd interaction that left the data plane active in FRR while the configuration layer silently reverted. Both failures are attributable to the verification environment rather than to diagnostic or reasoning gaps in

the agent.

3.2 Efficiency Outcomes

Session durations ranged from 2 minutes 47 seconds to 8 minutes 35 seconds, with a mean of 4 minutes 15 seconds and a median of 3 minutes 55 seconds. Total cost across all 50 sessions was $19.89, with a per-session mean of $0.40 and a median of $0.34. Per-session costs ranged from $0.23 to $1.27, with the upper tail driven by runs that adopted a file-based scripting strategy with higher tool call counts. Figure 5 displays the distribution of both metrics side by side, showing tight central clustering with a small number of high-duration, high-cost outliers corresponding to the file-based execution runs.

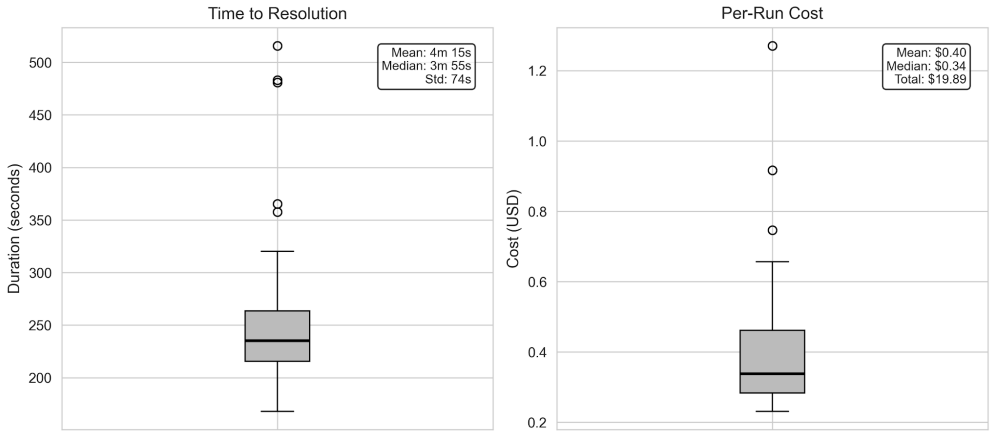


Figure 5: Time to Resolution and Per-Run Cost box plots

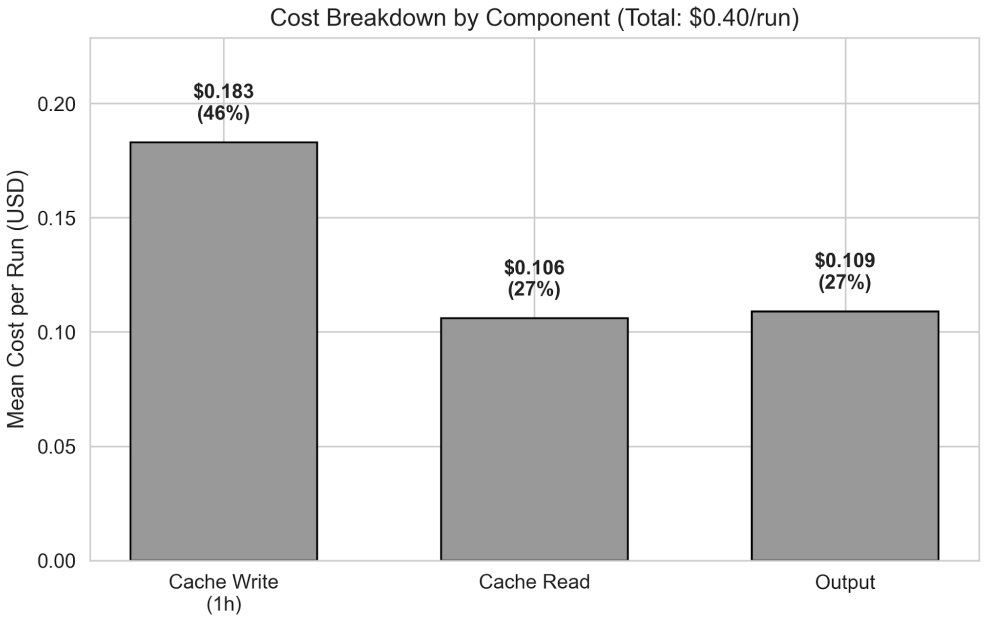


Figure 6: Cost Breakdown by Component

Figure 6 breaks down the per-session cost by token type. Cache write tokens accounted for the largest single share at approximately 46% (˜$0.18/run), with output tokens and cache read tokens each contributing approximately 27% (˜$0.11/run). This distribution reflects the structure of Claude Code's agentic loop: at each tool call turn, the full accumulated conversation history is re-submitted to the API, with newly appended content (primarily Netmiko command inputs and the router responses to those commands) written to a one-hour prompt cache. Previously cached content, including the stable `CLAUDE.md` system prompt and earlier tool exchanges, is retrieved as cache reads at one-tenth the standard input rate. Cache read tokens outnumbered write tokens by more than ten to one by volume (a mean of approximately 211,000 read tokens against 18,000 written per session), yet writes still accounted for the larger share of cost because the one-hour cache write rate is twenty times more expensive per token than the read rate ($10.00 vs. $0.50 per million tokens under Opus 4.6 pricing). Output tokens, despite averaging only 4,354 per session, matched cache reads in cost because they are billed at the highest rate of any token type ($25.00 per million). In practice, this means each additional tool call both extends the session cost through a

new write and incrementally grows the prefix that subsequent turns will read cheaply. Tool calls averaged 8.0 per session (median 7, range 3 to 37), with Bash accounting for 290 of the 402 total calls across all runs. The remaining calls consisted of 95 Write operations, 11 TaskOutput retrievals, 4 Edit operations, 1 ToolSearch, and 1 Read, reflecting the two distinct execution strategies described in the behavioral outcomes section.

3.3 Behavioral Outcomes

The agent exhibited a consistent diagnostic workflow across sessions but showed meaningful variation in execution method. Strategy phase analysis, derived from classifying each Bash command or script created and run as belonging to a diagnose, fix, or verify phase, shows a roughly balanced three-phase distribution. Diagnose commands accounted for a mean of 37.5% of Bash calls per session (median 40.0%), fix commands for 30.9% (median 28.6%), and verify commands for 31.6% (median 28.6%). Figure 7 plots this distribution across all 50 runs sorted by duration, showing that the three-phase balance remained broadly stable with no systematic drift toward diagnosis- or fix-heavy patterns as the experiment progressed.

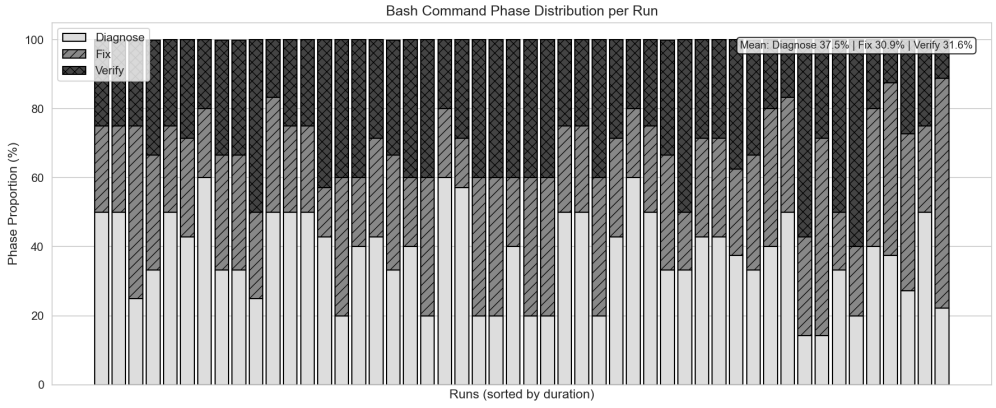


Figure 7: Bash Command Phase Distribution per Run

Rather than front-loading diagnosis, the agent interleaved substantial fix and verification activity throughout each session, reflecting a pattern of incremental correction with active confirmation at each step.

Device coverage data reinforces this interpretation. All three routers were touched in the diagnosis and verification phases across every one of the 50 sessions. In the fix phase, HQ and Branch2 were addressed in all 50 runs, while Branch1 was actively modified in only 8 of 50 runs. This is consistent with Branch1 hosting no injected faults: the agent consistently identified it as correctly configured during diagnosis and skipped it during the fix phase without error.

Two distinct execution strategies emerged across the 50 runs. Runs 1 through approximately 28 predominantly used inline Python via Bash, passing complete Netmiko scripts as `python3` heredocs without creating files on disk. Each inline script typically handled all three routers in a single loop, with a typical sequence of an inline diagnose script, an inline fix script, a brief sleep, and an inline verify script. This approach produced low tool call counts (3 to 7 per session) and lower per-session costs ($0.23 to $0.36). Beginning around run 29, the agent shifted to using the Write tool to create modular Python script files on disk (`gather.py`, `fix.py`, `verify.py`), then executing them with `python3`. This approach enabled parameterized, per-router execution and use of `run_in_background` for parallel diagnosis, but carried higher tool call counts (7 to 37) and higher per-session costs ($0.36 to $1.27). The 95 Write tool calls across all 50 runs are concentrated almost entirely in this later period.

The most consistent behavioral finding concerns Branch2. In 46 of 50 sessions, the agent's fix procedure left a residual `set protocols ospf interface eth2` entry not present in the answer key. This artifact arises from VyOS tree semantics: deleting the passive leaf under `interface eth2` leaves an empty parent tag node, which VyOS serializes as a bare `set` line until the subtree itself is deleted. Because it does not affect fault resolution and reflects platform-level behavior rather than an agent reasoning error, it is treated as a cosmetic divergence rather than a failure. No session across the 50 runs introduced a regression that left any fault in a worse state than it was found.

4 Conclusion

This study set out to determine whether a commercial, general-purpose coding agent could autonomously diagnose and resolve multi-fault OSPF scenarios in a traditional CLI-managed network environment, and to characterize the patterns of success and failure that emerge across repeated trials. Across 50 experimental sessions, Claude Code demonstrated a level of capability that warrants serious practitioner attention.

4.1 Summary of Findings

The effective fault resolution rate of approximately 97% (accounting for the functionally valid `default-information originate always` variant that the CCNA-derived rubric did not anticipate) establishes a meaningful demonstrated floor of capability for general-purpose coding agents in CLI-managed network environments. The corresponding perfect-run rate of approximately 78% reinforces that strong performance was the norm rather than the exception. The agent resolved faults on HQ and Branch2 consistently across nearly all sessions, correctly identified Branch1 as fault-free without instruction, and never required human intervention to navigate the CLI interaction model.

The most significant correctness finding, however, is not the fix rate but its complement: no session across the 50 runs introduced a regression that left any fault in a worse state than it was found. Given that the introduction explicitly frames regression as the failure mode of greatest operational consequence in network environments, specifically one that misleads operators about network state and can extend outages far beyond their original scope, this finding carries disproportionate weight. An agent that resolves faults at high rates while never making things worse presents a substantially different risk profile than one that achieves the same fix rate with occasional silent regressions.

The rubric mismatch on Error #3 deserves particular attention because it illustrates a tension

that will recur as agents are applied to operational domains with established certification-based answer keys. The agent's consistent selection of the `always` keyword reflects a legitimate engineering judgment (unconditional default route advertisement is arguably more resilient in scenarios where the upstream route may be intermittently absent) rather than a misunderstanding of the task. Strict evaluation against a single accepted command form would have scored this as a failure in 78% of runs. The implication for evaluation methodology is that correctness rubrics in network operations contexts must account for the existence of functionally equivalent configuration variants, or they will systematically undercount agent capability.

4.2 Efficiency and Behavioral Context

A mean time to resolution of 4 minutes 15 seconds and a mean per-session cost of $0.40 situate the agent within a range that is operationally credible for initial fault triage, particularly in smaller organizations where the alternative is waiting for a specialist with deep routing protocol expertise to become available. These figures are not presented as direct comparisons to human engineer performance, as no such comparison was in scope, but they establish that the agent's resource consumption is not a barrier to practical consideration.

The behavioral finding concerning the two distinct execution strategies (inline Python heredocs in the earlier sessions shifting to file-based modular scripts in the later sessions) is worth flagging for practitioners evaluating deployment consistency. While both strategies produced correct diagnostic and resolution outcomes, the shift in approach was not explicitly directed by the task prompt and represents a form of behavioral non-stationarity across sessions. Whether this reflects genuine variation in the agent's session-level reasoning or an artifact of the experimental sequence remains an open question, but it suggests that operators should not assume a fixed interaction pattern even when the task and environment are held constant.

The residual configuration artifact observed on Branch2 in 46 of 50 sessions is confirmed as operationally inert. When the agent deleted the passive leaf under `interface eth2`, VyOS's hierarchical configuration tree retained the now-empty parent node and serialized it as a bare `set protocols ospf interface eth2` line, a platform behavior that carries no routing consequence. The entry does not affect adjacency formation, route advertisement, or any other aspect of OSPF behavior; it is purely a tree hygiene issue. The fault was resolved correctly in all affected sessions. The finding does reinforce, however, that complete platform fluency would include knowing to delete the parent subtree rather than only the child leaf, a cleanup step a VyOS-experienced engineer would perform as a matter of habit.

The two sessions scoring 2/6 (r018 and r023) share an outcome that resists simple characterization: in both cases, the agent's diagnostic and remediation reasoning was correct, and the data plane confirmed it. Branch1's routing table in r018 showed the corrected 10.10.6.0/23 route via Branch2's eth2, a route that requires eth2 enabled, non-passive, and the correct network advertised, and ping tests from Branch2 to all destinations returned zero loss. Yet when the verification script read `show configuration commands` on Branch2 approximately two seconds after the session ended, all three original broken lines were present again. HQ's fixes, applied in the same session, survived into verification; only Branch2 reverted. In r023, a redundant `configure` token prepended to the Netmiko command list produced an `Invalid command` error but was cosmetically harmless; the remaining commands executed normally and HQ fixes committed correctly, while Branch2 exhibited the same revert pattern. The root cause cannot be confirmed post-hoc, but the most plausible explanation is a Netmiko/configd interaction that left the data plane active in FRR while the configuration layer silently reverted, producing a split state where routing behaved correctly but `show configuration` commands reflected the original broken config. What can be stated is that both failures are attributable to the verification environment rather than to agent reasoning, and that a simple post-commit re-read of `show configuration commands` (which the agent performed during diagnosis but not after applying fixes) would have either caught the revert

or confirmed the fix, producing a transcript consistent with the verifier's output either way.

4.3 Scope and Limitations

The external validity of these findings is bound by the experimental design in ways that must be stated plainly. The three-router VyOS topology is significantly smaller and less diverse than production enterprise environments, which routinely involve dozens to hundreds of devices across multiple vendors. The CCNA 11.6.3 fault set has a pedagogical origin and may be well-represented in the agent's training data in ways that inflate apparent performance relative to novel, production-realistic fault patterns. The study evaluated a single agent (Claude Code using the Opus 4.6 model at medium effort), and findings should not be generalized to other tools or model configurations without independent evaluation.

These limitations do not diminish the value of the findings, but they do define their scope. This study characterizes what a general-purpose coding agent can do in a controlled, well-specified CLI environment with a known fault set. Whether that capability transfers to larger, noisier, multi-vendor environments with unfamiliar fault patterns is a question that the present results motivate but do not answer.

4.4 Implications and Forward Direction

For practitioners, the primary takeaway is that commercial coding agents have crossed a capability threshold that makes them worth evaluating seriously for network fault triage, not as autonomous operators of production infrastructure, but as first-responders capable of rapidly diagnosing and resolving common protocol-layer faults in environments where deep routing expertise is not always immediately available. The zero-regression finding is particularly relevant to the question of supervised deployment: an agent that consistently avoids making things worse is a substantially safer candidate for human-in-the-loop triage assistance than one whose error modes are unpredictable.

For researchers, this study establishes a baseline and an evaluation methodology for a class of agent capability that has received no prior empirical treatment. The CLI interaction model used here (SSH-based terminal access with no programmatic interfaces) represents the operational reality for the majority of enterprise networks in production today, and the gap between that reality and the SDN-centric focus of most existing LLM network management research remains wide. Future work extending this evaluation to larger topologies, multi-vendor environments, and fault sets drawn from production incident logs would substantially strengthen the evidentiary basis for deployment guidance. Evaluation rubrics that account for functionally equivalent configuration variants will be essential to producing accurate capability assessments in this domain.

References

Adanza, D., Gifre, L., Ojaghi, B., Alemany, P., Muñoz, R., & Vilalta, R. (2026). A domain-specific autonomous agent for network traffic analysis. *Computer Networks*, *274*, 111809. https://doi.org/10.1016/j.comnet.2025.111809

Adanza, D., Gifre, L., Alemany, P., Natalino, C., Monti, P., Muñoz, R., & Vilalta, R. (2025). Leveraging generative AI for intent-based networking operations in network slices. *Computer Networks*, *272*, 111647. https://doi.org/10.1016/j.comnet.2025.111647

Duarte Ornelas, A. (2026). LLMnetopsLab [Retrieved April 20, 2026]. https://github.com/Doarte45/LLMnetopsLab

ITExamAnswers. (2023). 11.6.3 Lab – OSPF troubleshooting answers [Retrieved April 20, 2026]. https://itexamanswers.net/11-6-3-lab-ospf-troubleshooting-answers.html

Lin, S., Zhou, J., & Yu, M. (2025). An LLM-based agentic framework for accessible network control. *ACM SIGMETRICS Performance Evaluation Review*, *53*(2), 15–20. https://doi.org/10.1145/3764944.3764949

Merrill, M. A., Shaw, A. G., Carlini, N., Li, B., Raj, H., Bercovich, I., Shi, L., Shin, J. Y., Walshe, T., Buchanan, E. K., Shen, J., Ye, G., Lin, H., Poulos, J., Wang, M., Nezhurina, M., Jitsev, J., Lu, D., Mastromichalakis, O. M., ... Schmidt, L. (2026, January). Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. https://arxiv.org/abs/2601.11868

Wang, C., Scazzariello, M., Farshin, A., Ferlin, S., Kostić, D., & Chiesa, M. (2024). NetConfEval: Can LLMs facilitate network configuration? *Proceedings of the ACM on Networking*, *2*(CoNEXT2). https://doi.org/10.1145/3656296

Wei, Y., Xie, X., Hu, T., Zuo, Y., Chen, X., Chi, K., & Cui, Y. (2025). INTA: Intent-based translation for network configuration with LLM agents. *2025 IEEE 33rd International Conference on Network Protocols (ICNP)*, 1–16. https://doi.org/10.1109/ICNP65844.2025.11192391